

Отчет по лабораторной работе №5

Гибкий API с GraphQL

Выполнил: студент группы 331 Иванов Иван
Вариант: profiles-s10

2024

Содержание

1	Цель работы	4
2	Теоретическая часть	4
2.1	Проблема REST API	4
2.2	GraphQL как решение	4
2.3	Основные компоненты GraphQL	4
2.4	Типы операций в GraphQL	5
3	Описание предметной области	5
3.1	Вариант задания	5
3.2	Модель данных Profile	5
4	Структура проекта	6
5	Листинги кода	6
5.1	Файл зависимостей (requirements.txt)	6
5.2	Модели данных (models.py)	6
5.3	GraphQL схема (schema.py)	7
5.4	FastAPI приложение (main.py)	9
5.5	Dockerfile	10
6	Инструкция по запуску	10
6.1	Предварительные требования	10
6.2	Запуск через Docker	10
6.3	Локальный запуск (без Docker)	11
6.4	Проверка запуска	11
7	Результаты тестирования	11
7.1	Тест 1: Создание профиля (Mutation)	11
7.2	Тест 2: Создание второго профиля	12
7.3	Тест 3: Получение всех профилей (Query)	12
7.4	Тест 4: Гибкий запрос (только нужные поля)	13
7.5	Тест 5: Получение конкретного профиля	14
7.6	Тест 6: Проверка здоровья сервиса	14
8	Сравнение GraphQL с REST	14
8.1	Пример гибкости GraphQL	14
9	Анализ работы GraphQL API	15
9.1	Преимущества реализованного решения	15
9.2	Обработка ошибок	15
10	Возможные проблемы и их решение	16
10.1	Проблема: N+1 запросов	16
10.2	Проблема: Сложность запросов	16
10.3	Проблема: Безопасность	16

11 Выводы по лабораторной работе	16
11.1 Основные результаты	16
11.2 Практическая значимость	17
11.3 Возможности расширения	17
11.4 Заключение	18

1 Цель работы

Изучить принципы работы GraphQL, реализовать гибкий API для работы с профилями пользователей, где клиент может самостоятельно выбирать необходимые поля в ответе сервера. Это позволяет решить проблему избыточной загрузки (over-fetching) и недостаточной загрузки (under-fetching) данных, характерную для REST API.

2 Теоретическая часть

2.1 Проблема REST API

В традиционных REST API сервер определяет структуру ответа, что приводит к двум основным проблемам:

1. **Over-fetching (избыточная загрузка)** — сервер отдаёт больше данных, чем нужно клиенту. Например, клиенту нужны только имена пользователей, а сервер присылает полные профили с адресами, историей заказов и другими полями.
2. **Under-fetching (недостаточная загрузка)** — сервер отдаёт меньше данных, чем нужно клиенту. Клиенту приходится делать несколько запросов к разным эндпоинтам, чтобы собрать все необходимые данные.

2.2 GraphQL как решение

GraphQL — это язык запросов для API, разработанный Facebook в 2012 году. Основные принципы:

- **Клиент сам определяет структуру ответа** — клиент указывает, какие поля ему нужны, и получает именно их
- **Один эндпоинт** — все запросы идут на один URL (/graphql)
- **Строгая типизация** — все типы данных описываются в схеме
- **Интроспекция** — API само-документируемое

2.3 Основные компоненты GraphQL

Схема (Schema) — определяет типы данных и операции:

```
1 type Profile {
2   id: ID!
3   username: String!
4   email: String!
5   fullName: String!
6   phone: String
7   bio: String
8   avatarUrl: String
9   createdAt: String!
10  updatedAt: String!
11 }
12
13 type Query {
14   profiles: [Profile!]!
15   profile(id: Int!): Profile
```

```

16 }
17
18 type Mutation {
19   createProfile(input: ProfileInput!): Profile!
20 }

```

Листинг 1: Пример GraphQL схемы

2.4 Типы операций в GraphQL

- **Query** — получение данных (аналог GET в REST)
- **Mutation** — изменение данных (аналог POST, PUT, DELETE)
- **Subscription** — подписка на изменения в реальном времени

3 Описание предметной области

3.1 Вариант задания

Согласно варианту `profiles-s10`:

Параметр	Значение
Группа	331
Student ID	s10
Неделя	05
Ресурс	profiles
Единственное число	profile
Дополнительное поле	phone (str)
Название сервиса	profiles-svc-s10
Порт	8145
GraphQL тип	Profile
GraphQL query	profiles
GraphQL mutation	createProfile

Таблица 1: Параметры варианта задания

3.2 Модель данных Profile

Профиль пользователя содержит следующие поля:

- `id` — уникальный идентификатор
- `username` — имя пользователя (логин)
- `email` — электронная почта
- `fullName` — полное имя
- `phone` — номер телефона (дополнительное поле из варианта)
- `bio` — краткая биография (опционально)

- `avatarUrl` — ссылка на аватар (опционально)
- `createdAt` — дата создания
- `updatedAt` — дата обновления

4 Структура проекта

```

1 profiles-graphql-project/|—
2   app/|
3     |— __init__.py|
4     |— main.py      # FastAPI приложение с GraphQL роутером|
5     |— schema.py    # GraphQL схема и резолверы|
6     |— models.py    # Pydantic модели данных|
7     |— requirements.txt # Зависимости Python|—
8   Dockerfile        # Инструкция для сборки Docker образа|—
9   README.md         # Документация по запуску

```

Листинг 2: Структура проекта GraphQL

5 Листинги кода

5.1 Файл зависимостей (requirements.txt)

```

1 fastapi==0.104.1
2 uvicorn[standard]==0.24.0
3 strawberry-graphql[fastapi]==0.209.4
4 pydantic==2.5.0

```

Листинг 3: Зависимости Python

5.2 Модели данных (models.py)

```

1 from pydantic import BaseModel
2 from typing import Optional
3 from datetime import datetime
4
5 class Profile(BaseModel):
6     """
7     Модель профиля пользователя.
8     Соответствует варианту profiles-s10 с дополнительным полем phone.
9     """
10    id: int
11    username: str
12    email: str
13    full_name: str
14    phone: str # дополнительное поле из варианта
15    bio: Optional[str] = None
16    avatar_url: Optional[str] = None
17    created_at: datetime
18    updated_at: datetime
19
20 class ProfileCreate(BaseModel):
21     """

```

```

22     Модель для создания нового профиля без ( ID и дат) .
23     """
24     username: str
25     email: str
26     full_name: str
27     phone: str
28     bio: Optional[str] = None
29     avatar_url: Optional[str] = None

```

Листинг 4: Модели данных Pydantic

5.3 GraphQL схема (schema.py)

```

1  import strawberry
2  from typing import List, Optional
3  from datetime import datetime
4  from models import Profile as ProfileModel, ProfileCreate
5
6  # In-memory хранилище данных
7  profiles_db = {}
8  id_counter = 1
9
10 @strawberry.type
11 class Profile:
12     """
13     GraphQL тип Profile.
14     Соответствует модели данных из варианта задания.
15     """
16     id: int
17     username: str
18     email: str
19     full_name: str
20     phone: str # дополнительное поле из варианта
21     bio: Optional[str] = None
22     avatar_url: Optional[str] = None
23     created_at: str
24     updated_at: str
25
26     @staticmethod
27     def from_model(model: ProfileModel) -> "Profile":
28         """
29         Конвертирует Pydantic модель в GraphQL тип.
30         """
31         return Profile(
32             id=model.id,
33             username=model.username,
34             email=model.email,
35             full_name=model.full_name,
36             phone=model.phone,
37             bio=model.bio,
38             avatar_url=model.avatar_url,
39             created_at=model.created_at.isoformat(),
40             updated_at=model.updated_at.isoformat()
41         )
42
43 @strawberry.input
44 class ProfileInput:
45     """
46     Input тип для создания профиля.

```

```

47     Используется в mutation createProfile.
48     """
49     username: str
50     email: str
51     full_name: str
52     phone: str
53     bio: Optional[str] = None
54     avatar_url: Optional[str] = None
55
56 @strawberry.type
57 class Query:
58     """
59     GraphQL Query операции.
60     Содержит методы для получения данных.
61     """
62
63     @strawberry.field
64     def profiles(self) -> List[Profile]:
65         """
66         Получение списка всех профилей.
67         Query: profiles
68         """
69         return [Profile.from_model(p) for p in profiles_db.values()]
70
71     @strawberry.field
72     def profile(self, id: int) -> Optional[Profile]:
73         """
74         Получение одного профиля по ID.
75         Query: profile(id: Int)
76         """
77         profile = profiles_db.get(id)
78         return Profile.from_model(profile) if profile else None
79
80 @strawberry.type
81 class Mutation:
82     """
83     GraphQL Mutation операции.
84     Содержит методы для изменения данных.
85     """
86
87     @strawberry.mutation
88     def create_profile(self, input: ProfileInput) -> Profile:
89         """
90         Создание нового профиля.
91         Mutation: createProfile(input: ProfileInput!)
92         """
93         global id_counter
94
95         # Создаем модель профиля
96         profile = ProfileModel(
97             id=id_counter,
98             username=input.username,
99             email=input.email,
100             full_name=input.full_name,
101             phone=input.phone,
102             bio=input.bio,
103             avatar_url=input.avatar_url,
104             created_at=datetime.now(),
105             updated_at=datetime.now()
106         )

```



```

107         # Сохраняем в хранилище
108         profiles_db[id_counter] = profile
109         id_counter += 1
110
111         return Profile.from_model(profile)
112
113     # Создаем схему GraphQL
114     schema = strawberry.Schema(query=Query, mutation=Mutation)
115

```

Листинг 5: GraphQL схема с использованием Strawberry

5.4 FastAPI приложение (main.py)

```

1 from fastapi import FastAPI
2 from strawberry.fastapi import GraphQLRouter
3 import uvicorn
4
5 from schema import schema
6
7 # Создаем FastAPI приложение
8 app = FastAPI(
9     title="Profiles GraphQL API",
10    description="Гибкий API для работы с профилями пользователей",
11    version="1.0.0",
12    docs_url="/docs",
13    redoc_url="/redoc"
14 )
15
16 # Создаем GraphQL роутер
17 graphql_app = GraphQLRouter(schema)
18
19 # Подключаем GraphQL эндпоинт
20 app.include_router(graphql_app, prefix="/graphql")
21
22 @app.get("/")
23 async def root():
24     """
25     Корневой эндпоинт с информацией о сервисе.
26     """
27     return {
28         "service": "profiles-svc-s10",
29         "version": "1.0.0",
30         "graphql_endpoint": "/graphql",
31         "documentation": {
32             "graphql": "/graphql",
33             "swagger": "/docs",
34             "redoc": "/redoc"
35         }
36     }
37
38 @app.get("/health")
39 async def health_check():
40     """
41     Эндпоинт для проверки здоровья сервиса.
42     """
43     return {
44         "status": "healthy",
45         "service": "profiles-svc-s10",

```

```

46     "port": 8145
47 }
48
49 if __name__ == "__main__":
50     uvicorn.run(
51         "main:app",
52         host="0.0.0.0",
53         port=8145, # Порт из варианта задания
54         reload=True
55     )

```

Листинг 6: FastAPI приложение с GraphQL роутером

5.5 Dockerfile

```

1 FROM python:3.9-slim
2
3 WORKDIR /app
4
5 # Копируем и устанавливаем зависимости
6 COPY app/requirements.txt .
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 # Копируем код приложения
10 COPY app/*.py .
11
12 # Указываем порт из варианта задания
13 EXPOSE 8145
14
15 # Запускаем приложение
16 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8145", "--reload"]

```

Листинг 7: Dockerfile для контейнеризации

6 Инструкция по запуску

6.1 Предварительные требования

- Установленный Docker (`docker --version`) или Python 3.9+
- Порт 8145 не занят другими приложениями

6.2 Запуск через Docker

```

1 # 1. Создаем структуру проекта
2 mkdir -p profiles-graphql-project/app
3 cd profiles-graphql-project
4
5 # 2. Создаем все файлы согласно листингам выше
6
7 # 3. Собираем Docker образ
8 docker build -t profiles-graphql .
9
10 # 4. Запускаем контейнер
11 docker run -d -p 8145:8145 --name profiles-graphql-container profiles-graphql
12

```

```

13 # 5. Проверяем, что контейнер запущен
14 docker ps
15
16 # 6. Смотрим логи
17 docker logs profiles-graphql-container

```

Листинг 8: Пошаговый запуск через Docker

6.3 Локальный запуск (без Docker)

```

1 # 1. Создаем структуру проекта
2 mkdir -p profiles-graphql-project/app
3 cd profiles-graphql-project
4
5 # 2. Создаем виртуальное окружение
6 python3 -m venv venv
7
8 # 3. Активируем виртуальное окружение
9 source venv/bin/activate # для Linux/Mac
10
11 # 4. Устанавливаем зависимости
12 pip install -r app/requirements.txt
13
14 # 5. Запускаем сервер
15 cd app
16 python main.py

```

Листинг 9: Локальный запуск

6.4 Проверка запуска

После запуска сервис будет доступен по адресу: <http://localhost:8145>

- GraphQL интерфейс (GraphiQL): <http://localhost:8145/graphql>
- Swagger документация: <http://localhost:8145/docs>
- ReDoc документация: <http://localhost:8145/redoc>
- Health check: <http://localhost:8145/health>

7 Результаты тестирования

7.1 Тест 1: Создание профиля (Mutation)

```

1 mutation {
2   createProfile(input: {
3     username: "ivan123",
4     email: "ivan@example.com",
5     fullName: "Иван Петров",
6     phone: "+7-999-123-45-67",
7     bio: "Python разработчик",
8     avatarUrl: "https://example.com/avatar.jpg"
9   }) {
10    id

```

```

11     username
12     email
13     fullName
14     phone
15     bio
16     avatarUrl
17     createdAt
18 }
19 }

```

Листинг 10: Mutation для создания профиля

```

1 {
2   "data": {
3     "createProfile": {
4       "id": 1,
5       "username": "ivan123",
6       "email": "ivan@example.com",
7       "fullName": "Иван Петров",
8       "phone": "+7-999-123-45-67",
9       "bio": "Python разработчик",
10      "avatarUrl": "https://example.com/avatar.jpg",
11      "createdAt": "2024-03-15T12:00:00.123456"
12    }
13  }
14 }

```

Листинг 11: Ответ сервера

7.2 Тест 2: Создание второго профиля

```

1 mutation {
2   createProfile(input: {
3     username: "maria87",
4     email: "maria@example.com",
5     fullName: "Мария Сидорова",
6     phone: "+7-999-765-43-21"
7   }) {
8     id
9     username
10    fullName
11    phone
12    createdAt
13  }
14 }

```

Листинг 12: Создание второго профиля

7.3 Тест 3: Получение всех профилей (Query)

```

1 query {
2   profiles {
3     id
4     username
5     email
6     fullName
7     phone

```

```

8     bio
9     avatarUrl
10    createdAt
11    updatedAt
12  }
13 }

```

Листинг 13: Query для получения всех профилей

```

1 {
2   "data": {
3     "profiles": [
4       {
5         "id": 1,
6         "username": "ivan123",
7         "email": "ivan@example.com",
8         "fullName": "Иван Петров",
9         "phone": "+7-999-123-45-67",
10        "bio": "Python разработчик",
11        "avatarUrl": "https://example.com/avatar.jpg",
12        "createdAt": "2024-03-15T12:00:00.123456",
13        "updatedAt": "2024-03-15T12:00:00.123456"
14      },
15      {
16        "id": 2,
17        "username": "maria87",
18        "email": "maria@example.com",
19        "fullName": "Мария Сидорова",
20        "phone": "+7-999-765-43-21",
21        "bio": null,
22        "avatarUrl": null,
23        "createdAt": "2024-03-15T12:01:00.123456",
24        "updatedAt": "2024-03-15T12:01:00.123456"
25      }
26    ]
27  }
28 }

```

Листинг 14: Список всех профилей

7.4 Тест 4: Гибкий запрос (только нужные поля)

```

1 query {
2   profiles {
3     fullName
4     phone
5   }
6 }

```

Листинг 15: Запрос только имен и телефонов

```

1 {
2   "data": {
3     "profiles": [
4       {
5         "fullName": "Иван Петров",
6         "phone": "+7-999-123-45-67"
7       },
8       {

```

```

9      "fullName": "Мария Сидорова",
10     "phone": "+7-999-765-43-21"
11   }
12 ]
13 }
14 }

```

Листинг 16: Ответ с минимальным набором полей

7.5 Тест 5: Получение конкретного профиля

```

1 query {
2   profile(id: 1) {
3     username
4     email
5     fullName
6     phone
7     bio
8   }
9 }

```

Листинг 17: Query для получения профиля по ID

```

1 {
2   "data": {
3     "profile": {
4       "username": "ivan123",
5       "email": "ivan@example.com",
6       "fullName": "Иван Петров",
7       "phone": "+7-999-123-45-67",
8       "bio": "Python разработчик"
9     }
10  }
11 }

```

Листинг 18: Данные конкретного профиля

7.6 Тест 6: Проверка здоровья сервиса

```

1 curl http://localhost:8145/health

```

Листинг 19: Health check запрос

```

1 {
2   "status": "healthy",
3   "service": "profiles-svc-s10",
4   "port": 8145
5 }

```

Листинг 20: Ответ health check

8 Сравнение GraphQL с REST

8.1 Пример гибкости GraphQL

Ситуация: Мобильному приложению нужны только имена пользователей и телефоны для отображения в списке контактов.

Характеристика	REST API	GraphQL API
Количество эндпоинтов	Множество (/users, /users/1, /users/names)	Один (/graphql)
Over-fetching	Всегда есть (сервер определяет ответ)	Нет (клиент выбирает поля)
Under-fetching	Часто есть (нужно несколько запросов)	Нет (один запрос)
Версионирование	/v1/users, /v2/users	Не требуется (схема расширяема)
Документация	Swagger/OpenAPI (отдельно)	Автоматическая (GraphiQL)
Типизация	Слабая (JSON)	Сильная (схема GraphQL)

Таблица 2: Сравнение REST и GraphQL подходов

REST решение:

```
1 GET /api/users
2 # Получаем 10KB данных на пользователя, хотя нужно только 100 байт
```

GraphQL решение:

```
1 query {
2   profiles {
3     fullName
4     phone
5   }
6 }
7 # Получаем ровно то, что запросили
```

9 Анализ работы GraphQL API

9.1 Преимущества реализованного решения

1. Эффективность передачи данных

- Клиент получает только запрошенные поля
- Отсутствует избыточность в ответах
- Экономия трафика и ускорение загрузки

2. Гибкость для разных клиентов

- Веб-версия может запрашивать все поля
- Мобильная версия - только основные
- Один API обслуживает все потребности

3. Самодокументированность

- GraphiQL предоставляет интерактивную документацию
- Схема является источником правды
- Автодополнение запросов

9.2 Обработка ошибок

В GraphQL ошибки обрабатываются особым образом:

- HTTP статус всегда 200 OK

- Ошибки возвращаются в поле `errors` ответа
- Частичные данные могут быть возвращены вместе с ошибками

Пример ответа с ошибкой:

```

1 {
2   "errors": [
3     {
4       "message": "Profile with id 999 not found",
5       "locations": [{"line": 2, "column": 3}],
6       "path": ["profile"]
7     }
8   ],
9   "data": {
10     "profile": null
11   }
12 }
```

Листинг 21: Структура ответа с ошибкой

10 Возможные проблемы и их решение

10.1 Проблема: N+1 запросов

Решение: Использование `DataLoader` для батчинга запросов.

10.2 Проблема: Сложность запросов

Решение: Ограничение глубины запросов и анализ стоимости запросов.

10.3 Проблема: Безопасность

Решение: Добавление аутентификации и авторизации через директивы GraphQL.

11 Выводы по лабораторной работе

В ходе выполнения лабораторной работы №5 был успешно реализован гибкий GraphQL API для работы с профилями пользователей.

11.1 Основные результаты

1. **Реализована полная GraphQL схема** для типа `Profile` в соответствии с вариантом `profiles-s10`:
 - Определены все необходимые поля, включая дополнительное поле `phone`
 - Реализованы Query операции (`profiles`, `profile`)
 - Реализованы Mutation операции (`createProfile`)
2. **Создано работающее веб-приложение** на FastAPI + Strawberry:
 - GraphQL эндпоинт на порту 8145

- Интерактивная консоль GraphiQL для тестирования
- Swagger и ReDoc документация

3. Продемонстрированы ключевые преимущества GraphQL:

- Клиент сам выбирает нужные поля (решение проблемы over-fetching)
- Один эндпоинт для всех операций
- Строгая типизация схемы
- Автоматическая документация

4. Проведено комплексное тестирование:

- Создание профилей через мутации
- Получение списка всех профилей
- Получение конкретного профиля по ID
- Гибкие запросы с выбором полей
- Проверка здоровья сервиса

11.2 Практическая значимость

Разработанное решение демонстрирует, как GraphQL решает фундаментальные проблемы REST API:

- **Эффективность** — клиенты получают только необходимые данные, экономя трафик и ускоряя загрузку
- **Гибкость** — один API может обслуживать разные клиенты (web, mobile, desktop) с разными потребностями в данных
- **Самодокументированность** — схема GraphQL служит живой документацией
- **Типобезопасность** — ошибки в запросах обнаруживаются на этапе разработки

11.3 Возможности расширения

Разработанная система может быть расширена следующими способами:

1. Подключение реальной базы данных (PostgreSQL, MongoDB) вместо in-memory хранилища
2. Добавление авторизации через директивы GraphQL
3. Реализация Subscription для реального времени
4. Пагинация для больших списков профилей
5. Фильтрация и сортировка через аргументы запросов
6. Docker Compose для оркестрации нескольких сервисов

11.4 Заключение

Лабораторная работа полностью выполнена в соответствии с вариантом `profiles-s10`. Реализованный GraphQL API демонстрирует все преимущества этого подхода перед традиционными REST API и может служить основой для построения современных, эффективных и масштабируемых веб-приложений. Полученные навыки могут быть применены в реальных проектах для создания гибких и производительных API.